

ROBOTICS MASTER - CS-473

SYSTEM PROGRAMMING FOR SYSTEM ON CHIP



REPORT PW2

GROUP 24

Ashvin KUGATHASAN, Maxime LUYET

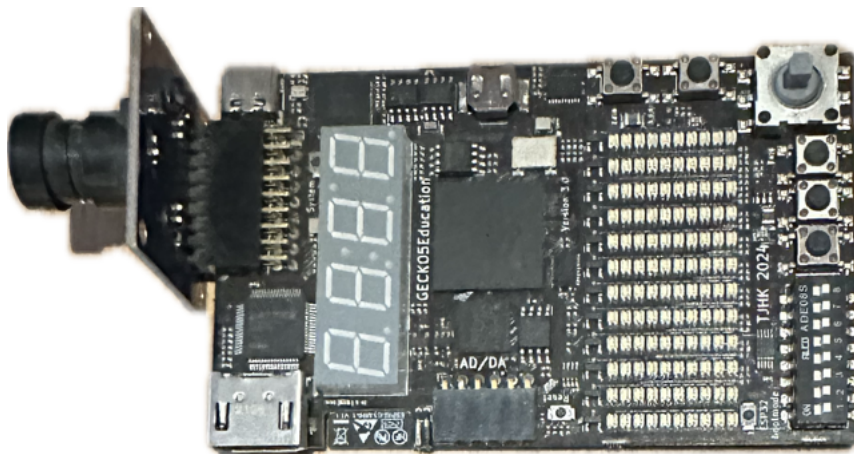


Figure 1: GECKO5 illustration

Contents

1 Introduction

1.1 Mandelbrot problem

2 Fixed point Mandelbrot

2.1 Fixed point format choice

2.2 Multiplication

2.3 Results

3 Custom Floating point Mandelbrot

3.1 Choice of the field widths

3.2 Implementation

3.3 Results

4 Conclusion

1 Introduction

This lab aims to replace the software floating-point library, used for computing the Mandelbrot set, with faster custom number representations. Begin with a fixed-point format, then a custom floating-point format. Both are evaluated on the virtual prototype of the GECKO5 board.

1.1 Mandelbrot problem

The Mandelbrot set is the set of complex numbers c for which the following recursive sequence remains bounded:

$$Z_n = \begin{cases} 0 & \text{for } n = 0, \\ Z_{n-1}^2 + c & \text{for } n \geq 1, \text{ with } c \in \mathbb{C}. \end{cases} \quad (1)$$

So, c belongs to the Mandelbrot set if $|Z_n| \not\rightarrow \infty$ as $n \rightarrow \infty$. In practice, it can be shown that the sequence diverges as soon as $|Z_n| \geq 2$. Each pixel is therefore iterated until this threshold is exceeded or until a maximum number of iterations n_{max} is reached. The number of iterations then determines the pixel's colour.

2 Fixed point Mandelbrot

The signed fixed-point format Qx.y stores a real number in a 32-bit two's complement integer, with x integer bits (sign included) and y fractional bits, as shown in Figure 2. A real value A is stored as the integer $A \cdot 2^y$.

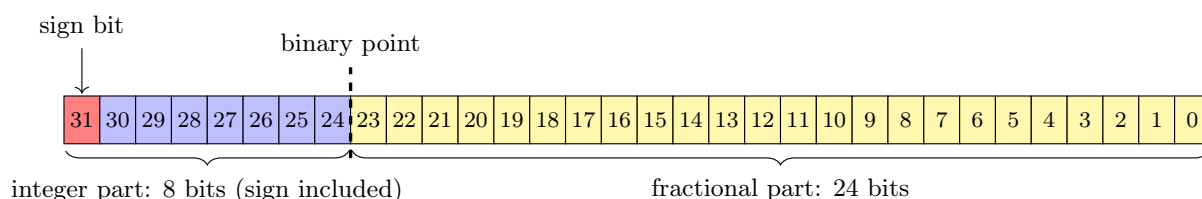


Figure 2: Q8.24 signed fixed-point format stored in a 32-bit two's complement integer.

The resolution of a Qx.y format is 2^{-y} and its range is $[-2^{x-1}, 2^{x-1}[$. Since $x + y = 32$, every bit given to the fractional part doubles the resolution but halves the range. The best format is therefore the one with the fewest integer bits whose range still covers every value produced by the algorithm without overflowing.

2.1 Fixed point format choice

Choosing a representation requires bounding every intermediate value of the algorithm. The viewing window bounds the inputs to $-2 \leq c_x < 1$ and $-1.5 \leq c_y < 1.5$. The sequence diverges mathematically as soon as $x^2 + y^2 \geq 4$, and a software "fast escape" mechanism bounds the operands by $|x| \leq 2$ and $|y| \leq 2$ strictly before the multiplications. Propagating these bounds gives:

$$\begin{aligned} x^2, y^2 &\in [0, 4], & 2xy &\leq 8, \\ x' = x^2 - y^2 + c_x &\leq 6, & y' = 2xy + c_y &< 9.5. \end{aligned}$$

Since no value exceeds 9.5, representing the whole range would call for Q5.27 and its range $[-16, 16[$. We nevertheless chose Q4.28, trading range for one additional bit of resolution, because the resulting overflow is provably harmless.

2.1.1 Overflow

The overflow stems from the position of the loop condition. `xx` and `yy` are computed at the top of the loop body and are not recomputed when x and y are overwritten at its end, so the condition authorises the next iteration on the basis of the *previous* state. The fast escape guard, meanwhile, tests each component separately and thus validates a square of side 4 rather than the disc of radius 2. A point in a corner of that square lies at a distance of up to $2\sqrt{2} \approx 2.83$ from the origin and passes unnoticed. The algorithm therefore performs exactly one iteration on a point that has already escaped, and it is that single iteration which drives y beyond 8.

Q4.28 represents only $[-8, 8[$, so such a value is not stored as itself but wrapped, as $y \mp 2^4$, so the range $[8, 9.5[$ is stored as $[-8, -6.5[$. This has no impact, because a wrapped value is never read. Indeed, $|y'| > 8$ implies $|2xy| > 8 - 1.5 = 6.5$, and the inequality property gives $2|xy| \leq x^2 + y^2$, which is exactly the quantity `xx + yy` evaluated by the loop condition. It therefore sees at least 6.5, well above the threshold of 4, and terminates on that same iteration. The corrupted value is discarded before it can influence anything, only the iteration count is returned, and it is correct.

Format	Integer bits	Range	Resolution
Q8.24	8	$[-128, 128[$	2^{-24}
Q5.27	5	$[-16, 16[$	2^{-27}
Q4.28	4	$[-8, 8[$	2^{-28}

Table 1: Candidate formats.

2.2 Multiplication

The product of two stored values is $(A \cdot 2^{28})(B \cdot 2^{28}) = AB \cdot 2^{56}$, which is a Q8.56 number on 64 bits. The OR1300 only provides a $32 \times 32 \rightarrow 32$ multiplication, so each operand is shifted right by 14 bits before multiplying:

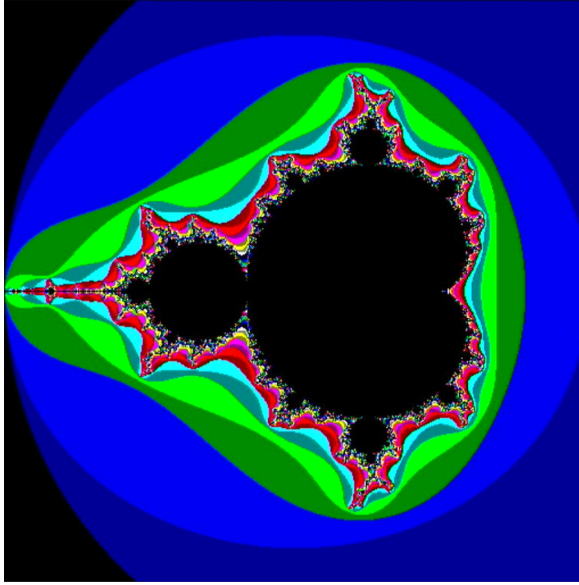
$$(a \gg 14) \cdot (b \gg 14), \quad 14 + 14 = 28,$$

and the result is directly in Q4.28. The escape bounds guarantee $|a| \leq 2$, that is 2^{29} in Q4.28, each shifted operand is below 2^{15} and the product is below 2^{30} , so no hardware overflow occurs during the multiplication. The two shifts must always add up to the number of fractional bits. The drawback is that each operand keeps only 14 fractional bits, so the effective precision of the product is half of the format's fractional bits.

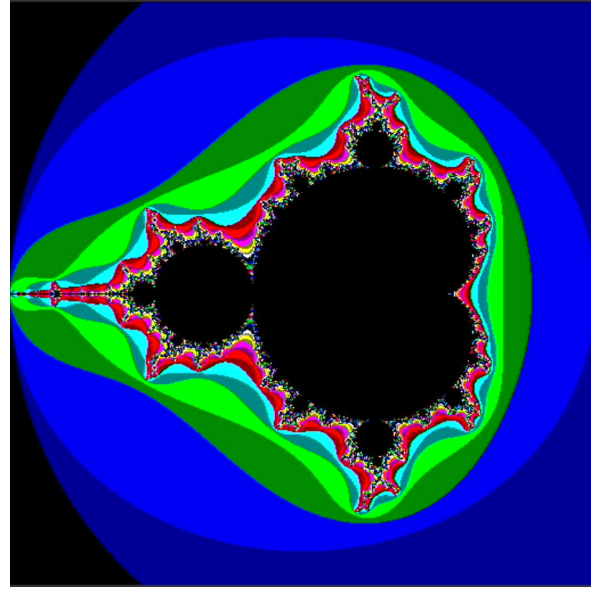
2.3 Results

The fixed-point rendering is visually similar to the floating-point reference, same outline of the set, same color bands, same band boundaries (Figure 3). The same image is produced in 8.95 s instead of 91.88 s.

The main limitation is the half-precision multiplication. Splitting each operand into 14-bit halves would compute the full product in 32 bits, at the cost of three multiplications instead of one. When zooming in, the 2^{-28} resolution would also become limiting once the pixel spacing drops below it.



(a) Floating-point, executed in 91.88 s



(b) Fixed-point (Q4.28), executed in 8.95 s

Figure 3: Comparison of the Mandelbrot set rendered with both representations.

3 Custom Floating point Mandelbrot

A floating-point number represents a real value R with a sign bit s , a mantissa m stored in fixed-point, and an exponent e stored with a bias (excess- k notation):

$$R = (-1)^s \cdot m \cdot 2^{e-k} \quad (2)$$

In the IEEE 754 single-precision format used by the `float` type, s is the MSB, e is coded on 8 bits in excess-127, and the mantissa $1 \leq m < 2$ is stored on 23 bits, its leading 1 being implicit. Unlike fixed-point, the position of the binary point moves with the exponent. This gives a much larger dynamic range with a constant relative precision.

This flexibility comes at a cost. To multiply two numbers, the mantissas are multiplied and the exponents added. To add them, the mantissas must first be aligned to the same exponent. In both cases, the result must then be normalized. Without an FPU, each of these steps is emulated in software, which explains the poor performance of the original implementation. Moreover, the generic library must also handle cases that never occur in the Mandelbrot algorithm, such as infinities, NaNs, or very large exponents. A custom format, tailored to the small value range of the algorithm, can therefore simplify these operations and make them faster.

3.1 Choice of the field widths

The mantissa fixes how many significant bits a value carries, the exponent the scale at which it sits. Widening the mantissa does not extend the range but improves precision uniformly, since a normalised mantissa carries the same number of significant bits whatever the magnitude.

Precision is what matters here. The Mandelbrot iteration is chaotic near the border of the set. A rounding error is amplified at every iteration until it changes the escape count, and hence the colour of the pixel. We therefore give the mantissa a sign and 23 bits of magnitude, in two's complement on 24 bits. The remaining 8 bits hold the exponent, in two's complement and without bias.

This is ample range. The escape test keeps $|x|, |y| \leq 2$ before every multiplication, so no intermediate exceeds about 10, and values only need to reach well below the pixel step $3/512 \approx 2^{-7.5}$. The representable range, roughly 2^{-106} to 2^{150} , covers this with a wide margin. A narrower mantissa, such as 15 bits, leaves visibly more wrong pixels along the border, while beyond 23 bits the gain is no longer visible. Conversely, too short an exponent flushes the small terms of the iteration to zero and degrades the image.

The cost lies in the multiplication, the product of two 23-bit magnitudes is 46 bits wide. Casting to 64 bits would call the software routine `__muldi3`, and truncating the operands first would fall back to 15 bits of precision, the very flaw of the fixed-point version. We instead split the product into three 32-bit partial products, as described below.

3.2 Implementation

Five choices keep the operations cheap.

Normalisation in one step. A value is brought back into the range $2^{22} \leq |m| < 2^{23}$ by locating its leading bit with `ilog2` and shifting once, rather than bit by bit. Working on the magnitude and reapplying the sign afterwards keeps negation exact. Values below the range are flushed to zero.

Exact multiplication in three instructions. Only the upper bits of the 46-bit product $P = u_a u_b$ are needed. With $u_b = b_h 2^{14} + b_l$ and $u_a = a_h 2^9 + a_l$, every partial product fits in 32 bits and

$$\lfloor P/2^{14} \rfloor = u_a b_h + ((a_h b_l + (a_l b_l \gg 9)) \gg 5)$$

holds exactly, for three `1.mul`. As both magnitudes lie in one octave, the result lies in $[2^{30}, 2^{32}[$, so the renormalising shift is 8 or 9, decided by testing bit 31. The squares use a variant whose sign is known.

Addition as an integer addition. With a two's complement mantissa, aligned values add as plain integers. Seven guard bits keep $|m| < 2^{30}$, so the sum cannot overflow, and preserve the bits that alignment would discard, which matters for the cancellation in $x^2 - y^2$. When the signs agree, the leading bit of the sum is bit 29 or 30, so one test replaces `ilog2`.

Comparison without arithmetic. Values are compared field by field, signs, then exponents, then mantissas. Thanks to normalisation, the exponent decides as soon as the signs agree, avoiding a subtraction and a normalisation for each of the three comparisons per iteration. The constants 2 and 4 are normalised at compile time.

Forced inlining. Compiled with `-O3`, GCC inlines almost nothing by itself. The operations are declared with the `__static_inline` macro of the support library, so the inner loop only calls `ilog2`, and only when an addition cancels.

Finally, doubling $2xy$ is a single integer addition, the exponent occupies the upper byte, so adding 2^{24} increments it without touching the mantissa. Since $|xy| \leq 4$, it stays far below its maximum.

3.3 Results

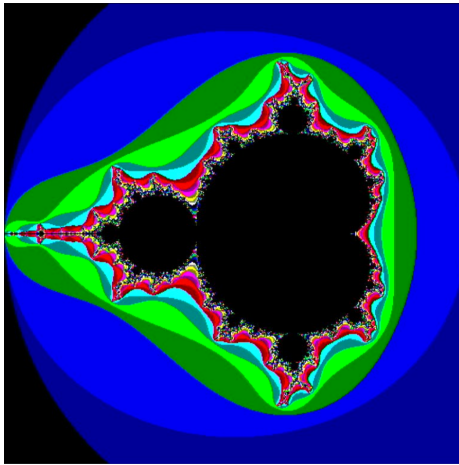
The visual output of the custom floating-point implementation remains identical to the standard software-emulated version as presented in Figure 4. The execution times are documented in

Table 2.

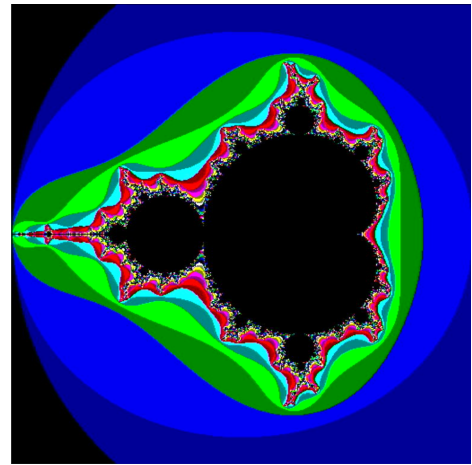
Version	Rendering time
<code>fractal_flpt</code> (emulated float)	91.88s
<code>fractal_myflpt</code> (custom float)	40.54 s

Table 2: Execution time comparison for custom floating-point implementation.

By entirely bypassing the compiler’s implicit floating-point libraries and replacing them with our custom inline `myflpt` operations, rendering time is vastly improved. While this approach is slower than pure fixed-point arithmetic (due to the required exponent alignment in `myflpt_add` and renormalization in `myflpt_mul`), it successfully provides the flexibility of a floating-point dynamic range without the massive instruction overhead of IEEE 754 software emulation.



(a) Floating-point.



(b) Custom floating point.

Figure 4: Comparison of the Mandelbrot set rendered with both representations.

4 Conclusion

This lab showed how strongly the choice of number representation determines performance on a processor without an FPU, where every IEEE 754 operation is emulated in software.

Bounding every intermediate value of the algorithm led to the Q4.28 fixed-point format. Its single overflow case was shown to be harmless, since the loop condition discards the wrapped value on the same iteration. Relying only on native integer operations, it renders the image in 8.95 s instead of 91.88 s, a $10.3\times$ speed-up. Its limit is precision, each operand of the multiplication keeps only 14 fractional bits.

The custom floating-point format addresses this limit. It uses an 8-bit exponent and a 24-bit two’s complement mantissa. An exact product is computed with three 32-bit multiplications, and additions reduce to plain integer additions. It reproduces the reference image in 40.54 s, a $2.3\times$ speed-up, keeping 23 significant bits whatever the magnitude. The remaining gap to fixed-point comes from operand alignment and normalisation, the price of a floating binary point.